

PANDUAN BELAJAR SOURCE CODE

Memahami Kode Sistem Pembiayaan Modal Syariah

Panduan pemula untuk memahami Java, JavaFX, FXML, CSS, Maven, dan alur Mudharabah/Musyarakah yang benar-benar ada di workspace.

Tujuan dokumen

Setelah membaca panduan ini, kamu diharapkan dapat menelusuri bagaimana aplikasi dibuka, bagaimana tampilan tersusun, bagaimana CSS diterapkan, bagaimana tombol terhubung ke Java, dan bagaimana data bergerak dari pengajuan sampai pencairan.

Disusun berdasarkan source proyek yang ada saat ini. Panduan menjelaskan kode dan perilaku yang benar-benar diterapkan, bukan rancangan fitur masa depan.

Peta belajar

1. Struktur folder dan tanggung jawab setiap bagian.
2. Urutan aplikasi mulai berjalan: Maven, JavaFX, FXMLLoader, FXML, CSS.
3. Dasar membaca FXML dan menghubungkan id komponen.
4. Penjelasan rinci syntax dan selector stylesheet `app.css`.
5. Peran controller serta hubungan event dan model.
6. Alur data: pengajuan, pemeriksaan, keputusan, akad, tanda tangan, pencairan.
7. Ringkasan class model, enum, OOP, batas sistem, dan latihan mengubah UI.

Cara menggunakan panduan: buka file proyek yang disebutkan sambil membaca. Jangan mulai dengan menghafal semua baris. Ikuti satu hubungan terlebih dahulu: FXML memberi id, controller mengambil id itu, lalu controller menjalankan aksi.

1. Peta folder workspace

```
java-project/
|-- pom.xml
|-- README.md
|-- src/
|   |-- main/
|       |-- java/
|           |-- Main.java
|           |-- enums/
|           |-- model/
|           |-- ui/
|       |-- resources/
|           |-- css/app.css
|           |-- views/*.fxml
|-- target/           (dibuat Maven ketika build)
```

Lokasi	Tanggung jawab	Yang biasanya kamu edit
src/main/java/ui	Peluncur JavaFX dan kode interaksi layar.	Perilaku tombol, validasi, format data, dan alur dialog.
src/main/resources/views	Struktur visual deklaratif dalam XML FXML.	Urutan kontrol, teks label, kolom tabel, layout.
src/main/resources/css	Aturan visual untuk warna, ukuran, jarak, garis, fokus dan hover.	Palet, tipografi, tombol dan gaya tabel.
src/main/java/model	Aturan domain pembiayaan; tidak bergantung pada tampilan.	Aturan akad, status, validasi, analisis, dan pencairan.
src/main/java/enums	Daftar nilai terbatas yang menjadi tipe status/jenis.	Tambah nilai hanya jika alur bisnis memang mendukungnya.
pom.xml	Konfigurasi Maven, versi Java, JavaFX dan plugin.	Dependencies/build; jangan mengubahnya untuk sekadar mengganti warna.
target	Hasil kompilasi dan salinan resource.	Jangan edit manual; akan dibuat ulang oleh Maven.

`Main.java` adalah demo berbasis console. GUI memakai `ui.BankSyariahApp` sebagai titik masuk. Keduanya memakai model yang sama, tetapi membentuk objek dan menjalankan alur dengan cara berbeda.

2. Dari klik Run sampai tampilan muncul

1. Perintah `.\mvnw.cmd javafx:run` meminta Maven Wrapper menjalankan plugin JavaFX.
2. Plugin menjalankan class utama `ui.BankSyariahApp` sebagaimana diatur di `pom.xml`.
3. `Application.launch` memulai runtime JavaFX, lalu JavaFX memanggil `start(Stage stage)`.
4. `FXMLLoader` membuka resource `/views/bank-syariah.fxml`, membuat node JavaFX dari markup, kemudian menghubungkan controller utama.
5. File root FXML menunjuk `ui.BankSyariahController` lewat `fx:controller`. `@FXML` mengizinkan injeksi node dan pemanggilan handler privat.
6. Scene memakai stylesheet `/css/app.css`. Selector CSS dicocokkan ke style class milik node.
7. Controller mengatur tabel, listeners dan tampilan awal. Stage menampilkan scene kepada pengguna.

Peran tiga lapisan UI

Lapisan	File	Analogi
Markup/layout	FXML	Denah: kontrol apa dan berada di dalam layout yang mana.
Presentasi/style	CSS JavaFX	Aturan rupa: warna, jarak, garis, ukuran dan state visual.
Interaksi	Controller Java	Menghubungkan input pengguna ke validasi dan aturan model.

Pemisahan penting: CSS tidak menyimpan data dan tidak memutuskan persetujuan. FXML tidak menghitung kelayakan. Controller mengatur interaksi, sedangkan keputusan/validasi domain seharusnya tetap dijaga model.

3. FXML: membaca struktur tanpa takut XML

Elemen dasar yang sering terlihat

```
<BorderPane styleClass="app-shell"
            xmlns="http://javafx.com/javafx/21"
            xmlns:fx="http://javafx.com/fxml/1"
            fx:controller="ui.BankSyariahController">
    <left>...</left>
    <center>...</center>
</BorderPane>
```

Syntax	Arti
<code><BorderPane></code>	Tag buka yang membuat object JavaFX BorderPane. Tag penutupnya mengakhiri elemen.
<code>xmlns</code>	Namespace XML untuk nama komponen JavaFX. Ini bukan CSS.
<code>xmlns:fx</code>	Mendaftarkan awalan khusus FXML, sehingga atribut <code>fx:controller</code> , <code>fx:id</code> dapat dikenali loader.
<code>fx:controller</code>	Nama class Java yang mengendalikan view root tersebut.
<code>styleClass</code>	Nama class CSS. Nilainya harus cocok dengan selector seperti <code>.app-shell</code> .
<code><left></code> , <code><center></code> , <code><top></code>	Slot milik BorderPane. Posisi anak ditentukan oleh slot, bukan koordinat piksel.
<code>text="..."</code>	Property kontrol, misalnya isi Label atau Button.
<code>onAction="#createApplication"</code>	Event tombol dipetakan ke method handler bernama itu di controller.

Layout = container bersarang

- `BorderPane` : area top/left/center/right/bottom. Layar utama menggunakannya untuk sidebar kiri dan workspace tengah.
- `VBox` : menumpuk anak vertikal. Cocok untuk judul lalu tabel, atau label di atas kontrol.
- `HBox` : menaruh anak horizontal. Dipakai untuk judul dan tombol, serta daftar dan panel detail berdampingan.
- `StackPane` : menumpuk node pada area yang sama. Di sini dua halaman berada pada tempat yang sama; controller mengubah `visible` dan `managed`.
- `GridPane` : susunan baris/kolom, terutama untuk pasangan label-input di form.
- `ScrollPane` : memberi scroll pada konten form yang lebih tinggi dari jendela dialog.

fx:id: kabel antara FXML dan Java

```
<TableView fx:id="employeeTable"/>
```

```
@FXML private TableView<CaseRecord> employeeTable;
```

Nama id harus sama persis, termasuk huruf besar/kecil. FXMLLoader menemukan kontrol yang diberi id dan memasukkannya ke field controller. Jika beda ejaan, field tidak terisi; pemakaian berikutnya bisa menghasilkan error.

Di formulir dialog ada pola tambahan: form tidak memiliki controller FXML tersendiri. Inner class `Form` memuat FXML, lalu mengambil kontrol dari namespace loader lewat `control("customerId", TextField.class)`.

4. Membaca layar utama

File `views/bank-syariah.fxml` menyusun layar dari luar ke dalam:

1. **Root BorderPane** bernama style class `app-shell`.
2. **Slot left** berisi VBox sidebar: merek, dua tombol navigasi, dan keterangan bahwa aplikasi demo tidak menyimpan data setelah ditutup.
3. **Slot center** berisi BorderPane workspace.
4. **Top workspace** berisi judul sistem dan label DEMO.
5. **Center workspace** berisi StackPane dengan halaman Nasabah dan Pegawai.
6. Setiap halaman menaruh heading di atas, lalu daftar/tabel dan detail pada satu baris.

Atribut layout yang menentukan keluwesan ukuran

Atribut	Makna praktis
<code>prefWidth</code> / <code>prefHeight</code>	Ukuran yang diinginkan, bukan jaminan ukuran akhir.
<code>minWidth</code> / <code>maxWidth</code>	Batas minimum/maksimum bila layout mengatur ukuran.
<code>spacing</code>	Jarak antar-anak langsung pada VBox/HBox.
<code>padding</code> dan <code>Insets</code>	Ruang di sisi dalam node: atas, kanan, bawah, kiri.
<code>HBox.hgrow="ALWAYS"</code>	Petunjuk layout agar anak HBox boleh mengambil lebar sisa.
<code>VBox.vgrow="ALWAYS"</code>	Petunjuk layout agar tabel/detail mengisi tinggi yang tersedia.
<code>visible</code>	Apakah node terlihat.
<code>managed</code>	Apakah layout menghitung node saat mengatur ruang. Menyembunyikan dan melepas managed mencegah halaman tersembunyi tetap memakan ruang.
<code>wrapText="true"</code>	Teks panjang berpindah baris alih-alih terus melebar.

Tombol navigasi menggunakan teks "Nasabah" dan "Pegawai", bukan karakter ikon dekoratif. Ini menghindari ikon yang tampil berbeda menurut font dan membuat fungsi kontrol eksplisit.

5. Form-form FXML dan tujuannya

FXML	Tujuan	Contoh kontrol / id penting
<code>application-form.fxml</code>	Input profil nasabah, usaha, rekening, data keuangan yang dilaporkan, serta rencana pembiayaan.	<code>customerId</code> , <code>monthlySalary</code> , <code>revenue</code> , <code>akad</code> , <code>bankShare</code> , <code>customerCapital</code> .
<code>financial-review-form.fxml</code>	Pegawai memeriksa angka usaha dan mencatat hasil pemeriksaan.	Empat angka, <code>cashFlowPreview</code> , <code>risk</code> , <code>verified</code> .
<code>decision-form.fxml</code>	Menampilkan indikator dan batas penilaian; pegawai memilih keputusan secara manual.	<code>profile</code> , <code>assessmentLimits</code> , <code>decision</code> , <code>approvedAmount</code> , <code>reason</code> .
<code>contract-form.fxml</code>	Memperlihatkan tanggal jatuh tempo yang dihitung dari tanggal pengajuan dan tenor.	<code>dueDate</code> .
<code>signature-form.fxml</code>	Mencatat tanggal tanda tangan akad.	<code>signatureDate</code> .
<code>disbursement-form.fxml</code>	Menampilkan jumlah/rekening tujuan, lalu meminta tanggal dan referensi transaksi.	<code>amount</code> , <code>account</code> , <code>date</code> , <code>reference</code> .

Contoh posisi label dan input dalam GridPane

```
<Label text="Nomor identitas" GridPane.rowIndex="1"/>
<TextField fx:id="customerId"
           GridPane.columnIndex="1" GridPane.rowIndex="1"/>
```

Label berada di kolom 0 dan baris 1 (kolom default adalah 0). TextField berada di kolom 1 pada baris yang sama. Baris dihitung mulai dari nol. `ColumnConstraints` memberi kolom label lebar minimum; kolom input mempunyai `hgrow="ALWAYS"` agar mengisi ruang yang tersedia.

FXML mendefinisikan susunan dan label, bukan validitas bisnis. Contohnya, teks petunjuk menyatakan total nisbah harus 100%, tetapi controller/model-lah yang benar-benar menolak total lain.

6. CSS JavaFX: cara membaca syntax

Contoh dari `css/app.css` :

```
.primary-button:hover {  
    -fx-background-color: #1d563b;  
}
```

- `.primary-button` adalah selector class. FXML menghubungkannya memakai `styleClass="primary-button"`.
- `:hover` adalah pseudo-class, yaitu keadaan sementara ketika pointer berada di atas tombol.
- `{ ... }` membungkus deklarasi yang berlaku untuk node terpilih.
- `-fx-` menandai property CSS milik JavaFX. CSS web standar tidak selalu berlaku identik.
- `property: value;` berarti nama aturan, titik dua, nilai, lalu titik koma.
- Warna hex `#1d563b` menyatakan RGB: dua digit merah, hijau, biru. `#FFFFFF` putih, `#000000` hitam.

Selector yang digunakan di proyek

Bentuk	Contoh	Artinya
Class tunggal	<code>.sidebar</code>	Node dengan style class sidebar.
Beberapa target	<code>.brand-caption, .sidebar-heading</code>	Aturan berlaku untuk kedua class; koma berarti "atau".
Class bertingkat/bersamaan	<code>.nav-button.nav-active</code>	Satu node yang memiliki kedua class sekaligus.
Anak di bawah parent	<code>.table-view .column-header .label</code>	Label di dalam header kolom di dalam TableView.
Pseudo-class	<code>.button:hover, .text-field:focused</code>	Gaya berubah berdasarkan keadaan hover/fokus.

Class bukan nama file dan bukan class Java. Ia adalah label visual pada node JavaFX. Satu node dapat memiliki lebih dari satu style class: class bawaan JavaFX dan class tambahan dari FXML.

7. Penjelasan stylesheet proyek: layout dan warna

Berikut pembacaan selector dan property yang benar-benar berada di `app.css`. Angka jarak memakai satuan CSS JavaFX yang secara praktis dibaca sebagai unit layout/pixel.

Aturan dasar dan kerangka

Selector / property	Arti dalam desain ini
<code>.root</code>	Aturan awal untuk root scene: seluruh aplikasi memakai Segoe UI, ukuran dasar 13, warna teks gelap, dan latar abu-hijau sangat pucat.
<code>.app-shell</code>	Warna dasar root layout utama; dibuat sama dengan latar root agar tidak terlihat ada bidang dengan warna berbeda.
<code>.sidebar</code>	Sidebar putih. Border transparan di tiga sisi dan garis kanan 1px memisahkannya dari area kerja tanpa bayangan dekoratif.
<code>.workspace</code>	Area isi memakai latar netral yang sama dengan root. Konten tidak dibungkus dekorasi tambahan.
<code>.topbar</code>	Header putih dengan padding 18 atas/bawah dan 24 kiri/kanan; garis bawah menjadi pemisah area.
<code>.page-view</code>	<code>-fx-spacing: 18px</code> memberi jarak antarblok utama satu halaman.
<code>.page-heading</code>	Jarak antarbagian di heading 16px.
<code>.content-row</code>	Daftar dan panel detail dipisahkan 16px.

Property visual yang berulang

Property	Makna	Contoh penggunaan
<code>-fx-font-family</code>	Font keluarga utama. Segoe UI lazim ada di Windows; bila tidak ada JavaFX memilih fallback font.	"Segoe UI"
<code>-fx-font-size</code>	Ukuran huruf. Hierarki dibangun dengan ukuran 11/12/13/14/17/22, bukan membesarkan semua teks.	11px untuk header tabel; 22px untuk judul halaman.
<code>-fx-font-weight</code>	Ketebalan font, misalnya <code>bold</code> untuk judul atau tombol utama.	Judul, label kolom, teks navigasi aktif.
<code>-fx-text-fill</code>	Warna teks JavaFX. Untuk teks, ini property yang tepat; bukan <code>color</code> CSS web.	Hijau gelap pada navigasi aktif.
<code>-fx-background-color</code>	Warna isi node atau komponen.	Putih untuk panel; hijau untuk CTA.
<code>-fx-border-color</code>	Warna garis tepi. Kata <code>transparent</code> membuat sisi tertentu tidak tampak.	Abu lembut untuk pemisah dan outline input.
<code>-fx-border-width</code>	Lebar garis untuk sisi atas, kanan, bawah, kiri.	<code>0 1px 0 0</code> berarti garis 1px hanya di kanan.

<code>-fx-padding</code>	Ruang di bagian dalam node; format empat angka: atas, kanan, bawah, kiri. Dua angka berarti atas/bawah lalu kiri/kanan.	<code>18px 24px</code> pada topbar.
<code>-fx-spacing</code>	Jarak antar-anak langsung di VBox/HBox.	Jarak daftar dan detail.
<code>-fx-background-radius</code>	Membulatkan latar.	Radius 3px pada kontrol: kecil, bukan pill di semua tempat.
<code>-fx-border-radius</code>	Membulatkan sudut border agar mengikuti latar.	Input dan panel radius kecil.
<code>-fx-alignment</code>	Posisi isi dalam kontrol.	<code>center-left</code> membuat teks tombol navigasi rata kiri.
<code>-fx-cursor</code>	Pointer berubah ketika hover; <code>hand</code> memberi isyarat bahwa elemen dapat diklik.	Tombol.
<code>-fx-size</code>	Ukuran header tabel JavaFX, dipakai di sini untuk tinggi header.	36px.
<code>-fx-cell-size</code>	Tinggi dasar baris tabel.	36px agar baris mudah dibaca.

8. Penjelasan stylesheet: komponen dan state

Teks, navigasi, dan hierarki

- `.brand-title` : nama BERKAH ukuran 17 dan bold; `.brand-caption` serta `.sidebar-heading` berbagi ukuran 11 dan warna teks sekunder.
- `.sidebar-heading` menambah bold pada label MENU, sehingga class gabungan tetap memakai aturan khusus ini di samping aturan bersama.
- `.sidebar-footer` menonjolkan teks "Demo tanpa login"; `.sidebar-note` memakai ukuran lebih kecil dan warna sekunder untuk penjelasan penyimpanan sementara.
- `.topbar-title` ukuran 14 bold; `.page-title` ukuran 22 bold untuk judul bagian; `.section-title` ukuran 13 bold untuk judul panel.
- `.muted-text` dan `.hint-text` memakai warna sekunder. Maksudnya hanya teks pendamping, bukan disabled state.
- `.environment-label` "DEMO" diberi garis dan huruf kecil bold. Bentuknya tag teks, bukan indikator status palsu.
- `.employee-identity` mengatur identitas pegawai demo dengan teks ukuran 12; ia sengaja bukan badge warna.

Tombol dan state pointer

- `.primary-button` adalah aksi terpenting: padding 9/13, hijau `#246747`, radius 3, teks putih bold, pointer hand.
- `.primary-button:hover` menggelapkan hijau menjadi `#1d563b` agar hover terasa tanpa animasi atau glow.
- `.button` umum menggunakan latar abu-hijau `#edf1ee` dengan teks gelap, agar aksi sekunder tidak bersaing dengan aksi utama.
- `.button:hover` mengubah latar sedikit lebih gelap.
- `.nav-button` rata kiri, transparan saat normal. `.nav-button:hover` memberi latar hover. `.nav-button.nav-active` baru berlaku jika dua class hadir; ia menandai halaman aktif dengan latar, teks, bold dan garis kiri hijau.

Input, tabel, dan fokus keyboard

- Selector gabungan `.text-field`, `.text-area`, `.choice-box`, `.date-picker` mengatur border abu, latar putih, radius kecil, dan padding untuk kontrol input.
- `.text-area` mendapat padding lebih kecil; `.text-area .content` menarget isi internal TextArea JavaFX agar permukaannya putih.
- `.table-view` punya border eksplisit. `.column-header-background` memberi header sedikit beda latar. `.column-header`, `.filler` mengatur tinggi dan garis bawah header.
- `.table-view .column-header .label` menata teks judul kolom; `.table-row-cell` menata tinggi dan pemisah tiap baris.
- `.table-row-cell:filled:selected` adalah baris yang memiliki isi dan dipilih. Seleksi memakai hijau pucat; `.text` di dalamnya menggunakan hijau tua.
- Selector `:focused` mengubah border komponen ketika fokus. Fokus penting untuk pengguna keyboard; menekan Tab harus tetap menunjukkan kontrol mana yang aktif.
- `.scroll-pane` dan `.scroll-pane > .viewport` menjaga scroll dialog tetap transparan agar latar scene/dialog terlihat konsisten.

Form dialog dan separator

- `.form-grid` menetapkan jarak kolom dan baris GridPane.
- `.form-section-title` menambah ruang sebelum judul kelompok field dan memakai hijau sebagai aksen teks.
- `.dialog-pane` mengatur permukaan dialog. Selector `>` berarti anak langsung: padding content dan button bar diberi ruang terpisah.
- `.separator .line` mengubah garis pemisah menjadi abu lembut.

Kontras: teks kecil sebaiknya mencapai 4.5:1 terhadap latar; batas komponen/fokus setidaknya 3:1. Warna sebaiknya dicek, tidak ditebak. Sebagian warna teks sekunder perlu dites di atas semua latar yang benar-benar dipakai.

9. Controller: penghubung antara UI dan aturan

Class `BankSyariahController` memiliki tiga kelompok tugas:

1. Menerima klik dan perubahan selection/input.
2. Membaca nilai input, memvalidasi, dan memanggil operasi model.
3. Memperbarui table/detail dan memberi pesan sukses atau kesalahan.

Arti syntax JavaFX yang penting

Syntax	Makna
<code>import ...</code>	Menggunakan class dari package lain tanpa menulis nama panjangnya berulang.
<code>package ui;</code>	Menempatkan class dalam namespace UI. Path file sesuai folder <code>ui</code> .
<code>@FXML</code>	Anotasi agar FXMLLoader boleh mengisi field atau memanggil handler, termasuk anggota <code>private</code> .
<code>private</code>	Hanya class terkait yang boleh mengakses anggota.
<code>final</code>	Referensi/field tidak bisa ditugasi ulang setelah diinisialisasi; isi object masih dapat berubah bila object-nya mutable.
<code>void</code>	Method tidak mengembalikan nilai.
<code>if / else</code>	Memilih cabang berdasarkan kondisi, contohnya jenis akad atau tahap proses.
<code>try / catch</code>	Menangani error yang diketahui secara eksplisit dan menampilkan pesan kepada pengguna.
<code>lambda event -> ...</code>	Fungsi ringkas untuk listener/handler, dijalankan saat event terjadi.
<code>this::label</code>	Method reference: menunjuk sebuah method sebagai fungsi yang akan dipanggil kemudian.
<code>StringBuilder</code>	Menyusun teks detail panjang secara bertahap menggunakan <code>append</code> .

Field UI dan model tabel

Field seperti `@FXML private TableView<CaseRecord> employeeTable` berarti TableView menampilkan item bertipe CaseRecord. Generics di dalam `<...>` membantu compiler menjaga tipe.

`ObservableList` adalah daftar yang dapat memberi tahu komponen ketika isinya berubah. `FilteredList` menyeleksi daftar untuk pencarian; `SortedList` mendukung urutan tabel. Dua sisi mempunyai list wrapper masing-masing, tetapi berangkat dari kumpulan record yang sama.

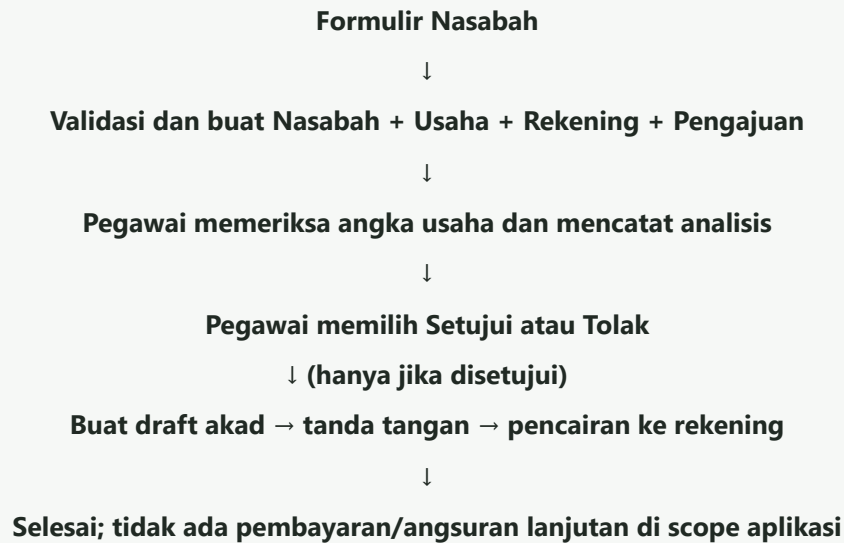
`setCellValueFactory` memberi tahu kolom bagaimana mengubah satu CaseRecord menjadi teks kolom. Contoh kolom Nominal mengambil pengajuan lalu memanggil `rupiah(...)`. `selectedItemProperty().addListener(...)` berarti perubahan baris terpilih akan memanggil method yang menampilkan detail.

Mengapa ada CaseRecord di controller?

`CaseRecord` mengikat objek Nasabah, Usaha, Rekening, Pengajuan, dan keputusan UI menjadi satu baris pilihan. Ini bukan tabel database; ia record sementara di memori agar kedua table view bisa menunjuk objek kasus yang

sama.

10. Alur lengkap satu pengajuan



Langkah 1 — buat pengajuan

`createApplication()` memuat `application-form.fxml`, mengambil komponen berdasarkan `fx:id`, lalu mengatur `TextFormatter` angka. Saat pengguna menyimpan, method validasi memeriksa field wajib, tanggal lahir, nominal/tenor positif, pilihan akad, total nisbah 100%, dan modal nasabah khusus Musyarakah. Baru setelah validasi, object domain dibuat dan dimasukkan ke daftar records.

`form.show(() -> {...})` membuka dialog dengan callback validasi. Jika validasi melempar `IllegalArgumentException`, tombol simpan dibatalkan, dialog tetap terbuka, dan pesan kesalahan ditampilkan.

Langkah 2 — analisis pegawai

`recordAnalysis(record)` membuka form review dengan angka yang dilaporkan nasabah sebagai nilai awal, bukan angka fiktif. Listener memperbarui pratinjau arus kas. Pegawai perlu mencentang konfirmasi dan menulis catatan pemeriksaan. Analisis mencatat nilai yang diperiksa.

```
Arus kas tersedia
= omzet bulanan
- biaya langsung
- biaya operasional
- kewajiban usaha
```

Pratinjau hanya indikator internal. Ia bukan skor kredit dan tidak memutuskan persetujuan. Tidak ada bukti dokumen, riwayat kredit, pengeluaran rumah tangga, atau jadwal pembayaran.

Langkah 3 — keputusan manusia

`decide(record)` menampilkan profil, keterbatasan analisis dan rekomendasi. Pegawai memilih keputusan secara eksplisit. Jika menyetujui, jumlah persetujuan harus diisi; jika menolak, jumlah disetujui dibuat 0. Alasan keputusan wajib.

Langkah 4–6 — akad, tanda tangan, pencairan

- `createContract` hanya tersedia untuk pengajuan yang disetujui; jatuh tempo dihitung dengan menambahkan tenor ke tanggal pengajuan.
- `signContract` menolak tanggal tanda tangan sebelum keputusan atau pada/setelah jatuh tempo.
- `disburse` menolak tanggal pencairan sebelum tanda tangan, meminta referensi transaksi, lalu memanggil method model akad untuk mengkredit rekening.

Controller menggambar tombol tahap berikutnya sesuai status. Ini mencegah lompatan alur; validasi yang sama juga dijaga di model, sehingga mengubah UI saja tidak dapat memintas semua aturan bisnis.

11. Model domain, status, dan OOP

Class/enum	Peran ringkas
BankSyariah	Bank dan pegawai yang terdaftar.
PegawaiBank	Pegawai yang menjalankan analisis dan menetapkan keputusan.
Nasabah	Data identitas, pekerjaan, gaji yang dilaporkan, usaha, rekening dan pengajuan.
Usaha	Data usaha milik nasabah yang menjadi objek pengajuan.
RekeningNasabah	Nomor dan saldo; menerima kredit ketika pencairan berhasil.
PengajuanPembiayaan	Data permintaan, angka laporan, status, riwayat, analisis, keputusan melalui akad.
AnalisisKelayakan	Snapshot angka bulanan yang diperiksa, rumus laba/arus kas, catatan dan rekomendasi internal.
KeputusanPembiayaan	Keputusan final pegawai, jumlah (atau 0 bila ditolak), alasan dan waktu.
PembuatAkad	Factory yang memilih object akad subtype berdasarkan JenisAkad.
AkadPembiayaan	Class abstrak yang menjaga keadaan draft, tanda tangan dan pencairan.
AkadMudharabah , AkadMusyarakah	Implementasi jenis akad; Musyarakah juga memegang modal nasabah.
Pencairan	Catatan transaksi pencairan yang menghubungkan akad, rekening, tanggal dan referensi.
RiwayatPengajuan	Node riwayat berantai; setiap node menyimpan status/waktu/pelaku/keterangan dan tautan ke node berikut.
JenisAkad	MUDHARABAH atau MUSYARAKAH .
StatusPengajuan	DIAJUKAN , DIPROSES , DISETUJUI , DITOLAK .
StatusKeputusan , StatusAkad	Nilai status terbatas untuk keputusan dan siklus akad.

Konsep OOP yang bisa ditemukan di kode

- **Encapsulation:** field private; perubahan status melalui method seperti `tandatangani` dan `cairkan` , bukan assignment bebas dari layar.
- **Inheritance:** AkadMudharabah dan AkadMusyarakah mewarisi AkadPembiayaan.
- **Abstraction:** AkadPembiayaan adalah abstract class dan meminta subclass menyediakan `getJenisAkad()` .
- **Polymorphism:** kode dapat menyimpan hasil factory sebagai AkadPembiayaan, walau object sebenarnya subtype Mudharabah/Musyarakah.
- **Enum:** pilihan status/akad dibatasi pada nilai yang didefinisikan; bukan string bebas yang mudah typo.
- **Composition of behavior:** Pengajuan mendelegasikan pembuatan akad ke PembuatAkad; keputusan tetap ditetapkan pegawai.

Linked list riwayat

Setiap RiwayatPengajuan menunjuk node berikutnya. Pengajuan menyimpan node pertama dan terakhir. Untuk membaca semua riwayat, kode mulai dari node pertama dan mengulang selama node bukan `null`, lalu maju ke node berikut. Ini konsep struktur data, bukan database atau audit log semua aksi.

12. Maven dan dependencies

`pom.xml` adalah file konfigurasi XML Maven (bukan Java). Bagian `properties` menetapkan target Java 21 dan versi JavaFX. Dependencies menyediakan `javafx-controls`, `javafx-fxml`, dan JUnit 5 pada scope test. Plugin compiler menyusun source; Surefire menjalankan tes; JavaFX Maven Plugin menjalankan `ui.BankSyariahApp`.

```
.\mvnw.cmd javafx:run    # menjalankan GUI
.\mvnw.cmd test          # compile dan mencoba tes
.\mvnw.cmd clean test    # hapus hasil target lalu build ulang
```

Folder `src/test` saat ini tidak ada. Jadi `test` tetap melakukan compile, tetapi Maven melaporkan "No tests to run". Build sukses bukan berarti perilaku UI sudah diuji otomatis.

13. Kalau ingin mengubah sesuatu, mulai dari sini

Tujuan	File / tempat	Urutan perubahan
Ganti warna tombol/teks/background	<code>css/app.css</code>	Ubah value CSS; cari selector yang cocok seperti <code>.primary-button</code> .
Atur jarak/ukuran panel	<code>css/app.css</code> dan/atau FXML	Style visual di CSS; batas/pref size dan nesting di FXML.
Ubah urutan layar dan tabel	<code>views/bank-syariah.fxml</code>	Pindah atau susun node. Jangan ganti/hapus fx:id tanpa memperbarui controller.
Tambah input pada dialog	FXML form + <code>BankSyariahController</code>	Tambah kontrol dengan fx:id, ambil kontrol di controller, parse/validasi, simpan ke model bila field itu data domain.
Ubah aturan nisbah/status/akad	<code>model</code> dan <code>enums</code>	Ubah aturan domain dahulu; sesuaikan controller hanya untuk menampilkan atau mengambil input.
Ubah versi JavaFX/library	<code>pom.xml</code>	Periksa kompatibilitas JDK, JavaFX dan plugin sebelum mengubah.

Langkah aman setelah edit

1. Pastikan semua nama `fx:id` yang dipakai controller masih cocok.
2. Jalankan `.\mvnw.cmd clean test` untuk kompilasi bersih.
3. Jalankan GUI dengan `.\mvnw.cmd javafx:run`.
4. Uji jalur nyata: data kosong, input tidak valid, batal dialog, pilih baris, pencarian, setiap status, dan tahap selanjutnya.
5. Jangan menganggap CSS berhasil hanya karena compile sukses. CSS bisa memiliki selector yang tidak cocok tanpa membuat Java gagal compile; periksa langsung saat aplikasi berjalan.

14. Batasan dan istilah yang perlu diingat

Istilah	Penjelasan
FXML	Markup XML untuk membuat hierarchy node JavaFX; FXMLLoader mengubahnya menjadi object saat runtime.
CSS JavaFX	Stylesheet dengan property JavaFX berawalan <code>-fx-</code> .
Controller	Class Java yang menerima event, membaca control, dan memperbarui tampilan.
Model / domain	Object dan aturan bisnis pembiayaan.
Event handler	Method yang dipanggil saat event seperti klik terjadi.
Listener	Fungsi yang mengamati perubahan property, misalnya teks pencarian atau selection tabel.
Resource path	Path classpath seperti <code>/views/bank-syariah.fxml</code> ; berbeda dari path folder Windows.
Factory	Object/method yang memilih dan membuat subtype yang diperlukan; di sini <code>PembuatAkad</code> .
Validasi	Pemeriksaan input atau keadaan sebelum operasi diterima. Controller memberi pesan cepat; model menjaga invariant domain.

Hal yang sengaja belum dilakukan sistem

- Tidak ada login atau pemisahan hak akses; menu hanya mengganti perspektif layar.
- Data disimpan di memory, bukan database/file. Menutup aplikasi menghapus data sesi.
- Tidak ada verifikasi dokumen pendapatan, credit bureau, pengeluaran pribadi atau perhitungan jadwal angsuran.
- Rekomendasi arus kas bukan skor kredit dan bukan keputusan otomatis.
- Alur selesai pada pencairan; tidak mengelola pembayaran, bagi hasil pascapencairan, tunggakan, atau penyelesaian akad.

Ringkasan utama: FXML menjelaskan apa yang tampak; CSS menjelaskan bagaimana tampilannya; controller menjelaskan apa yang terjadi ketika pengguna berinteraksi; model menjaga data dan aturan pembiayaan.

Panduan ini mencerminkan isi workspace pada saat dibuat. Jika source berubah, contoh dan penjelasan perlu diselaraskan kembali.